

DRD · SPARK HACK TORONTO

CivicVox-Omni

Design & Architecture Reference Document

Version: 1.0 — DRAFT **Date:** May 29, 2026

1. System Architecture Overview

CivicVox-Omni is composed of four loosely coupled layers, each independently buildable by a single team member. All layers communicate through clearly defined JSON contracts over local HTTP/WebSocket connections. No component requires internet access during operation.

Layer	Technology Stack	Owner
Local AI Inference Engine	NVIDIA TensorRT-LLM / vLLM / NVIDIA NIM, Faster-Whisper, Kokoro TTS	Person 1 (Infrastructure)
Agent Orchestration Engine	Python, LangGraph, LangChain, GeoPandas, ChromaDB	Person 2 (Agent & Data)
API Gateway	Python FastAPI, Uvicorn, WebSocket, Pydantic	Person 1 (Infrastructure)
Frontend Dashboard	Next.js 14, TypeScript, Tailwind CSS, Framer Motion, Mapbox GL JS	Person 3 (Frontend)

2. Layer 1 — Local AI Inference Engine

This layer is the computational core of CivicVox-Omni. It must be set up on the ASUS GX10 first, as every other layer depends on its endpoints.

2.1 Inference Server

Primary: NVIDIA NIM Containers

- Pull the NVIDIA NIM container image for meta/llama-3.2-11b-vision-instruct from nvcr.io.
- Expose an OpenAI-compatible REST endpoint at `http://localhost:8000/v1`.
- Compile the model using TensorRT-LLM for GB10 tensor core utilization (FP8 or INT4 quantization).

Fallback: vLLM or Ollama

- vLLM: `pip install vllm && python -m vllm.entrypoints.openai.api_server --model meta-llama/Llama-3.2-11B-Vision-Instruct`
- Ollama: `ollama serve && ollama pull llama3.2-vision`
- Both expose the same OpenAI-compatible API format — no agent code changes required.

2.2 Speech Processing Pipeline

Speech-to-Text (STT)

- Library: faster-whisper (pip install faster-whisper)

- Model: large-v3, loaded in float16 on CUDA
- Exposed as a utility function `transcribe_audio(pcm_bytes) -> str` within the FastAPI server.
- Target latency: < 300 ms for 10-second audio clip on GB10.

Text-to-Speech (TTS)

- Library: kokoro-onnx (pip install kokoro-onnx sounddevice)
- Model: Kokoro-82M (82M parameter, ultra-fast ONNX inference)
- Exposed as `synthesize_speech(text: str) -> bytes` returning a WAV audio buffer streamed to the frontend.

2.3 Model Configuration Summary

Model	Task	Quantization	VRAM Est.	Latency Target
Llama-3.2-11B-Vision-Instruct	Vision analysis + text generation	INT4 / FP8	~8 GB	< 60 ms/token
Whisper Large-v3	Speech-to-text	FP16	~3 GB	< 300 ms / 10 s audio
Kokoro-82M	Text-to-speech	FP32 ONNX	< 1 GB	< 100 ms / sentence

3. Layer 2 — Agent Orchestration Engine

This layer implements the multi-agent state machine using LangGraph. It contains all business logic, system prompts, and geospatial query functions. It is self-contained in a single Python module (`agents/civic_vox_graph.py`) and imports from a shared data module (`data/toronto_loader.py`).

3.1 State Schema

The LangGraph StateGraph uses a typed Python TypedDict to track all data flowing between agents:

AgentState fields

messages (list[BaseMessage]) · video_frames_base64 (list[str]) · gps_coordinates (dict) · vision_analysis (dict) · spatial_data_results (dict) · next_step (str) · final_dispatch_report (str)

3.2 Agent Nodes

Node 1: Orchestrator Agent

- Entry point of the graph. Receives the transcribed voice input and decides routing.

- System prompt directs it to output a single JSON object: { requires_vision: bool, urgency_level: 'LOW' | 'HIGH' | 'CRITICAL' }.
- If requires_vision is true, routes to Vision Agent; otherwise routes directly to Localizer Agent.
- Strips markdown fences from model output before JSON parsing (open-weight models frequently wrap JSON in ```json ... ``` blocks).

Node 2: Vision Agent

- Receives the latest base64-encoded video frame and the current transcription.
- Sends a multimodal message to the local VLM endpoint: a text instruction + image_url content block.
- System prompt instructs the model to output only: { hazard_type, severity_scale (1-10), water_depth_m, structural_risk, location_cues }.
- Stores parsed dict in state.vision_analysis and advances to Localizer Agent.

Node 3: Localizer Agent (GIS / Open Data RAG)

- Reads state.gps_coordinates and executes a spatial query against in-memory GeoDataFrames.
- Calls get_closest_hydrants(lat, lng, n=3) — returns list of dicts with id, distance_meters, status.
- Calls get_building_specs(lat, lng) — reverse-geocodes against RentSafeTO data to find the nearest building record.
- Stores results in state.spatial_data_results and routes to Compiler node.

Node 4: Report Compiler

- Receives vision_analysis, spatial_data_results, and the original message from state.
- Constructs a synthesis prompt instructing the VLM to generate a structured dispatch protocol covering: threat classification, perimeter radius, infrastructure access nodes, building vulnerabilities, and recommended crew type.
- Stores the final string in state.final_dispatch_report and routes to END.

3.3 Graph Routing Logic

Conditional edges are implemented via a route_decision(state) function that reads state.next_step:

From Node	Condition	To Node
orchestrator	next_step == 'vision'	vision
orchestrator	next_step == 'localizer'	localizer
vision	always	localizer
localizer	always	compiler
compiler	always	END

3.4 Toronto Open Data Module (data/toronto_loader.py)

- Executed once at application startup; loads all GeoJSON files from /data/ into memory as GeoDataFrames.
- Builds an R-tree spatial index on the hydrants GeoDataFrame for sub-millisecond nearest-neighbour lookup.
- Projects all GeoDataFrames from WGS84 (EPSG:4326) to NAD83 / UTM Zone 17N (EPSG:2958) for accurate metre-based distance calculations.
- Exposes two public functions: `get_closest_hydrants(lat, lng, n)` and `get_building_specs(lat, lng)`.

4. Layer 3 — API Gateway

The API Gateway (backend/server.py) is a FastAPI application that acts as the bridge between the frontend and the agent engine. It is the only network-accessible component and runs on `http://0.0.0.0:8080`.

4.1 Endpoints

Method	Endpoint	Payload	Response
POST	/api/incident	{ transcript: str, frame_b64: str, gps: {lat, lng} }	{ report: str, spatial: dict, vision: dict }
WebSocket	/ws/stream	Binary audio chunks + JSON metadata frames	Server-sent JSON events with agent state updates
POST	/api/synthesize	{ text: str }	WAV audio bytes (Content-Type: audio/wav)
GET	/api/health	—	{ status: 'ok', models_loaded: bool }

4.2 Request Lifecycle

- Frontend sends a POST to /api/incident with the latest video frame (base64), transcribed text, and GPS coordinates.
- FastAPI validates the payload via Pydantic model and injects it into an AgentState dict.
- `civic_vox_engine.invoke(initial_state)` is called synchronously (or async via `ainvoke` for WebSocket mode).
- The completed state is serialised and returned as a JSON response.
- For WebSocket mode, server-sent events fire after each node completes, enabling live agent telemetry on the dashboard.

4.3 CORS & Local Network Configuration

- CORS origins set to `["*"]` for hackathon demo (mobile phone on same Wi-Fi network must reach the GB10's LAN IP).

- Server bound to 0.0.0.0:8080 — team members must note the GB10's local IP address (e.g., 192.168.x.x) for frontend environment variable configuration.

5. Layer 4 — Frontend Dashboard

The frontend is a Next.js 14 (App Router) application scaffolded with create-next-app. It runs on `http://localhost:3000` and communicates with the FastAPI backend over the local network.

5.1 Page Layout

The application uses a full-viewport dark grid (background #050F14) divided into three primary zones:

- Left panel (60% width): Mapbox GL JS dark-matter tile map, full-height.
- Right panel top (40% width, 50% height): Agent telemetry pipeline — four node cards (Orchestrator, Vision, Localizer, Compiler) with Framer Motion pulsing indicators.
- Right panel bottom (40% width, 50% height): Streaming dispatch report text window with typewriter rendering.
- Floating overlay (bottom-left of map): Live camera feed iframe / video element (200x150 px) showing the mobile stream.

5.2 Mapbox GL JS Integration

- Dark-matter style: `mapbox://styles/mapbox/dark-v11`.
- Initial center: `[-79.3832, 43.6532]` (Toronto City Hall), zoom 13.
- Incident layer: A red pulsing circle marker at the reported GPS coordinate, rendered via a GeoJSON source + circle paint layer with animated radius via `requestAnimationFrame`.
- Asset layers: Blue triangle markers for hydrant locations; orange building outline polygons for the RentSafeTO building footprint — both sourced from the API response JSON.
- Alert radius: A semi-transparent red fill circle layer centred on the incident, radius derived from the dispatch report's perimeter recommendation.

5.3 Live Camera Capture

- Component: `<CameraCapture />` — calls `navigator.mediaDevices.getUserMedia({ video: true, audio: true })`.
- Captures a JPEG snapshot every 2 000 ms using an offscreen canvas `drawImage()`, converts to base64, and stores in React state.
- Audio is captured as a `MediaRecorder` blob, converted to PCM, and POSTed alongside the image frame.
- On each capture cycle, a new `POST /api/incident` request is triggered automatically if an active incident is flagged.

5.4 Agent Telemetry Panel

- Four <AgentCard /> components rendered in a horizontal pipeline.
- Each card tracks one of: idle | active | complete | error states.
- Active state: pulsing Framer Motion ring animation (scale 1.0 → 1.15 → 1.0, 0.8 s loop, ease-in-out).
- Complete state: green checkmark icon fades in; card border transitions from teal to green.
- State transitions driven by WebSocket events from the backend (or parsed from the POST response for fallback polling mode).

5.5 Environment Variables

Variable	Value	Note
NEXT_PUBLIC_API_URL	http://<GB10_LAN_IP>:8080	Set to GB10's LAN IP before demo
NEXT_PUBLIC_MAPBOX_TOKEN	pk.eyJ1Ijojoi...	Use a free Mapbox public token
NEXT_PUBLIC_WS_URL	ws://<GB10_LAN_IP>:8080/ws/stream	WebSocket endpoint for live telemetry

6. Component Integration Map

The following table defines the exact JSON contract between each layer boundary. All teams must agree on these schemas before Hour 3 of the hackathon.

Interface	Direction	Schema / Format
Frontend → API Gateway	POST /api/incident	{ transcript: string, frame_b64: string, gps: { lat: number, lng: number } }
API Gateway → Agent Engine	Python function call	AgentState TypedDict (see Section 3.1)
Agent Engine → API Gateway	Return value	Completed AgentState dict
API Gateway → Frontend	JSON response	{ report: string, vision: VisionOutput, spatial: SpatialOutput, urgency: string }
Backend → Frontend (WS)	Server-sent JSON	{ node: 'orchestrator' 'vision' 'localizer' 'compiler', status: 'active' 'complete', timestamp: ISO8601 }
Inference Server → Agent Engine	OpenAI-compatible HTTP	POST /v1/chat/completions with messages array; response.choices[0].message.content

7. Developer Setup Guide

7.1 Repository Structure

Project Layout

```
/civicvox-omni /backend server.py      # FastAPI entry point  agents/    civic_vox_graph.py
# LangGraph state machine  data/    toronto_loader.py  # GeoDataFrame loader & spatial
queries    *.geojson      # Toronto Open Data files /frontend  app/    page.tsx      # Main
dashboard  components/    CameraCapture.tsx    AgentCard.tsx    MapView.tsx
DispatchReport.tsx  README.md  docker-compose.yml  # Optional: containerise FastAPI
```

7.2 Backend Setup

- Python 3.11+ required.
- `pip install langgraph langchain-core langchain-openai geopandas shapely fastapi uvicorn faster-whisper kokoro-onnx pydantic`
- Set `LOCAL_LLM_URL` environment variable to the running inference server (default: `http://localhost:8000/v1`).
- Run: `uvicorn backend.server:app --host 0.0.0.0 --port 8080 --reload`

7.3 Frontend Setup

- Node 20+ required.
- `cd frontend && npm install`
- Copy `.env.local.example` to `.env.local` and fill in GB10 LAN IP and Mapbox token.
- `npm run dev` — runs on `http://localhost:3000`

7.4 Data Download Script

- Run `python backend/data/download_toronto_data.py` to fetch all four Open Data GeoJSON files into `/backend/data/`.
- Total download size: approximately 150–300 MB depending on dataset versions.
- Pre-download before the hackathon to avoid reliance on venue Wi-Fi.

8. Demo Script & Pitch Talking Points

When presenting to NVIDIA and Antler judges, frame the demonstration around these key narrative beats:

Moment	What to Show	What to Say
Open with the problem	Static slide: city-wide emergency, cloud goes down	"During a city emergency, cloud latency is a liability. We eliminate it entirely."
Live demo start	Open phone camera, point at a staged flooding scene	"This is a citizen reporting live. No app install, just a browser."
Vision fires	Show Vision Agent card pulse → complete, map pin drops	"The GB10 just analyzed that frame locally — no data left this room."
Spatial query	Show hydrant markers appear on map	"Three nearest operational hydrants, surfaced from Toronto Open Data in under 20 ms."
Dispatch report streams	Show typewriter text in report panel	"A military-grade dispatch protocol, generated at edge speed."
Close with the chip	Show GPU utilization dashboard	"This is the NVIDIA GB10 doing what no cloud can: real-time multimodal AI at the edge, for the city."